

SchwarMAX — Agent's Guide

A hand-off document for an LLM helping a new user get started with the `schwarmax_pkg` codebase. Read this in full before your first task.

1. What SchwarMAX is

JAX-based Schwarzschild dynamical modelling framework for barred galaxies. Given IFU-style stellar kinematics (Voronoi-binned surface density Σ , mean line-of-sight velocity V , dispersion σ , and Gauss-Hermite moments h_1 – h_4) it reconstructs the joint constraint on:

- an NFW dark-matter halo,
- a Miyamoto-Nagai stellar disc,
- a T3 ellipsoidal bar + V4 ovoid bulge (Dehnen 2023),
- three viewing angles (α, β, γ), a mass-to-light ratio, and a bar pattern speed Ω .

The forward model integrates orbits in the bar's rotating frame, projects each orbit into IFU bins and Gauss-Hermite moments, then solves a non-negative-least-squares (NNLS) problem for the orbital weights that best reproduce the data. That inner NNLS + orbit-integration block runs **for every parameter proposal in an MCMC** (typically BlackJAX adaptive random-walk Metropolis on GPU).

The public interface lives in `schwarmax_pkg/` at the root of this repo. Everything is pure functions of `(theta, dict_data)` — no mutable globals, no runtime side effects.

2. How to use this document

If you are an LLM agent working with a user who wants to *use* SchwarMAX:

1. Read Sections 3–5 to internalise the package layout and parameter conventions.
2. Skim Sections 6–7 (entry points + workflows) — refer back when the user asks how to do something.
3. **Read Section 14 (pitfalls) carefully.** Every item is a real trap that has caused real bugs. Do not skip.
4. When you need to write code, prefer the pattern from Section 7 over inventing your own. If you have to invent, verify signatures with `Read` on the referenced file:line — do not guess.
5. The `example/` folder is the canonical end-to-end demonstration (Sections 9–10). Point the user there.

If the user asks something you can't answer from this document, read the relevant source file directly. Every function is short (< 200 lines) and well-named.

3. Directory layout

```
schwarmax_pkg/
├── constants.py
├── utils.py
├── potentials.py
├── dehnen_bar.py
├── jeans.py
├── ghmoments.py
├── integrates.py
├── nnls.py
├── model.py
└──
# <-- put on sys.path for `from utils import ...`
# G, KPCGYR_TO_KMS, EPSILON, HYPERGEOM tables
# data loader, param converters, rotation curves
# NFW, MN potential + density
# T3, T4, V4 potentials + densities
# Jeans-moment initial-condition sampler
# Gauss-Hermite basis + h_to_V_sigma
# adaptive leapfrog integrator (vmap+chunked)
# ADMM NNLS solver, single + bootstrap
# build_model, get_model_with_orbit,
# model_deltaChi2_jackknife ← main entry
```

```

└─ likelihood.py          # model_likelihood, calculate_delta_chi2
└─ CylindricalSpline.py  # cyl-spline potential expansion (bar branch)
└─ CubicSpline.py        # helper for CylindricalSpline
└─ example/
  └─ example_schwarmax.ipynb # run an MCMC
  └─ example_analysis.ipynb  # analyse a finished MCMC
  └─ mcmc_checkpoint.pkl      # 12-D BlackJAX chain (1000×32×12)
  └─ mock_Nbody_galaxy_beta65_gamma45_D50.pkl # Voronoi kinematic data
  └─ Bar_model_TG21/model/    # N-body snapshot + AGAMA .ini
└─ docs/
  └─ LLM_GUIDE.md            # this file

```

Runtime dependencies: jax, jaxopt, numpy, scipy, matplotlib, corner, agama, astropy, blackjax (only if training an MCMC).

4. Parameter convention

The public parameter vector is **12-D**. Enforce this order everywhere:

slot	name	unit	notes
0	logM_10kpc	log ₁₀ (M _☉)	NFW mass inside 10 kpc
1	logM_disc	log ₁₀ (M _☉)	MN disc mass
2	logM_bar	log ₁₀ (M _☉)	T3 bar mass = V4 bulge mass
3	logc_halo	log ₁₀	NFW concentration (Δ=200, pc=277.54)
4	logRs_disc	log ₁₀ (kpc)	MN scale length
5	logHs_disc	log ₁₀ (kpc)	MN scale height (= T3 b_bar)
6	logL_bar	log ₁₀ (kpc)	T3 half-length; a_bar = L_bar/5
7	alpha	radians	viewing angle α
8	beta	radians	viewing angle β
9	gamma	radians	viewing angle γ
10	log_mass_to_light	log ₁₀ (M/L)	see pitfall §14.1
11	log_Omega_bar	log ₁₀ (kpc/Gyr)	multiply by KPCGYR_TO_KMS for km/s/kpc

Angles inside the parameter vector are **radians**. `build_model` receives the baryon dict with angles converted to **degrees**. The `unpack_params` helper in `example/example_analysis.ipynb` does this conversion for you (Section 7.3 below).

An **older 13-D convention** exists in the sibling `SchwarMAX/` codebase. `example_analysis.ipynb` handles both — see pitfall §14.1.

5. dict_data schema

`utils.load_data_bootstrap(path, filename, N_BOOTSTRAP=100, n_samples=5000)` loads a mock Voronoi-binned pickle and returns a dict with the keys below. **Do not hand-craft dict_data — always use this loader.**

Kinematic data + errors:

key	shape	meaning
-----	-------	---------

XY_density_data	(n_bins,)	surface density Σ (in luminosity units)
XY_density_data_err	(n_bins,)	1% relative error
V_data, V_data_err	(n_bins,)	V _{los} and its error
sigma_data, sigma_data_err	(n_bins,)	σ and its error
h1_data .. h4_data	(n_bins,)	Gauss-Hermite moments
h1_data_err .. h4_data_err	(n_bins,)	GH moment errors
v0, s	(n_bins,)	per-bin base-Gaussian centre and width
num_per_bin	(n_bins,)	pixel count per Voronoi bin
bin_mapping	(n_pixels,)	pixel-index $\rightarrow$ bin-index (+1 sentinel at end)
total_bins	scalar (int)	n_bins

Voronoi grid geometry:

key	shape	meaning
X_regular_grid, Y_regular_grid	(n_pixels,)	pixel centres (kpc)
X_minmax, Y_minmax	(2,)	grid extent
nX_nY	(2,)	pixel count per axis
dX, dY	scalar	pixel size

Rzphi integration grid (deterministic, built inside the loader):

key	shape	meaning
R_grid, z_grid, phi_grid	(Rzphi_n_tot,)	(R, z, ϕ) cell centres
R_minmax, z_minmax, phi_minmax	(2,)	ranges
Rzphi_n_tot	scalar	10·6·10 = 600 by default
Rzphi_n_grid	(3,)	(n _R , n _z , n _{ϕ}) = (10, 6, 10)
dR, dz, dphi	scalars	cell widths
sample_for_integration	(1024, 3)	Sobol samples for sub-cell integration
sample_for_integration_XY	(1024, 3)	same for the XY grid

Bootstrap-perturbation normals (each has row 0 zeroed = unperturbed):

key	shape
XY_standard_normal	(N_BOOTSTRAP, n_bins)
h1_standard_normal .. h4_standard_normal	(N_BOOTSTRAP, n_bins)
V_standard_normal, sigma_standard_normal	(N_BOOTSTRAP, n_bins)

Initial-condition particles:

key	shape	meaning
-----	-------	---------

w0	(n_samples, 3)	(x, y, z) draws from the analytic disc
----	-------------------	---

n_samples defaults to 5,000 but the example notebooks use 7,500 to match production runs. Larger = better orbital library resolution.

6. Entry points and full signatures

6.1 `utils.load_data_bootstrap`

```
def load_data_bootstrap(path, filename, N_BOOTSTRAP=100, n_samples=5_000):
    # returns dict_data as in $5
```

6.2 `utils.logMenc_logc_to_logM_logRs, logM_logRs_to_logMenc_logc`

```
def logMenc_logc_to_logM_logRs(logM_enc, log_c,
                               r_enc=10.0, Delta=200., rho_crit=277.54):
    # (logM(<r_enc), log c) <-> (log M_NFW, log R_s)
def logM_logRs_to_logMenc_logc(logM_halo, logRs_halo,
                               r_enc=10.0, Delta=200., rho_crit=277.54):
    # inverse
```

Slot 0 of θ is `logM_10kpc`, slot 3 is `logc_halo` — the forward model wants (logM, logRs), so every wrapper starts with a call to `logMenc_logc_to_logM_logRs`.

6.3 `model.build_model` — forward model for χ^2 /likelihood

```
def build_model(density_func, potential_func,
                params_halo_pot, params_disk_rho, dict_data, num_Vbin,
                Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                Rzphi_lim_grid=jnp.array([[0,10],[-3,3],[-pi,pi]]),
                xy_lim_grid=jnp.array([[-10,10],[-3,3]]),
                xy_n_grid=jnp.array([60,40]),
                nnls_maxiter=200, regularization=1.0)
    -> dict[str, jnp.ndarray]
```

Returns a dict with these keys — all fields except `density_3d` and `weights` are **stacked over N_BOOTSTRAP bootstrap replicates** (row 0 = unperturbed):

key	shape	notes
<code>density_3d</code>	(Rzphi_n_tot,)	analytic 3-D density target (data)
<code>density_3d_model</code>	(N_boot, Rzphi_n_tot)	model 3-D density per bootstrap
<code>surface_density</code>	(N_boot, n_bins)	luminosity units
<code>V, sigma</code>	(N_boot, n_bins)	derived from h_1, h_2 via <code>h_to_V_sigma</code>
<code>h1..h4</code>	(N_boot, n_bins)	clipped to ± 10
<code>weights</code>	(N_boot, n_orb)	non-negative orbital weights

Static args (JIT recompiles when changed): `density_func, potential_func, num_Vbin, Rzphi_n_tot, nnls_maxiter.`

Time on A100: ~1 s per call after JIT warm-up. On CPU: ~60–100 s.

6.4 `model.get_model_with_orbit` — forward model + full orbit trajectories

```
def get_model_with_orbit(density_func, potential_func,
                        params_halo_pot, params_disk_rho, dict_data, num_Vbin,
                        Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                        Rzphi_lim_grid=jnp.array([[0,10],[-3,3],[-pi,pi]]),
                        xy_lim_grid=jnp.array([[-10,10],[-3,3]]),
                        xy_n_grid=jnp.array([60,40]),
                        nnls_maxiter=200, regularization=1.0)
    -> dict[str, jnp.ndarray]
```

Same forward model as `build_model`, but with `n_realizations=1` (one orbit per particle, no random jitter) and the integrator stores every accepted timestep's 6-D phase-space coordinates. Extra keys:

key	shape	notes
<code>weights</code>	<code>(n_particles,)</code>	unperturbed NNLS weights (row 0 of bootstrap)
<code>weights_all_boot</code>	<code>(N_boot, n_particles)</code>	per-bootstrap weights
<code>y_traj</code>	<code>(n_particles, 1, N_max, 6)</code>	<code>(x, y, z, vx, vy, vz)</code> in kpc, kpc/Gyr
<code>t_traj</code>	<code>(n_particles, 1, N_max)</code>	integration times in Gyr
<code>rotation_matrix</code>	<code>(3, 3)</code>	Z-X-Z rotation for viewing angles
<code>Omega_bar</code>	scalar	pattern speed in kpc/Gyr
<code>mass_per_orbit</code>	scalar	mean orbit mass (Msun)

Accepted timesteps are those with `dt = t[i+1]-t[i] > 0`. Padded positions (integrator finished before `N_max`) have `dt == 0` and should be masked out.

Same static-arg rules as `build_model`. Same runtime cost.

6.5 `model.model_deltaChi2_jackknife` — delete-d jackknife

```
def model_deltaChi2_jackknife(density_func, potential_func, chi2_func,
                              params_halo_pot, params_disk_rho, dict_data,
                              num_Vbin,
                              Rzphi_n_tot=360, Rzphi_n_grid=jnp.array([10,6,6]),
                              Rzphi_lim_grid=..., xy_lim_grid=..., xy_n_grid=...,
                              nnls_maxiter=200, regularization=1.0,
                              n_groups=100, batch_size=50)
    -> scalar (jnp.std of  $\chi^2$  across n_groups replicates)
```

`chi2_func` **must** have the signature:

```
def chi2_func(y_Rzphi, y_Rzphi_model, sigma_Rzphi,
              y_xy, y_xy_model, sigma_xy, # <-- ALL in luminosity units
              y_h1, y_h1_model, sigma_h1,
              y_h2, y_h2_model, sigma_h2,
              y_h3, y_h3_model, sigma_h3,
              y_h4, y_h4_model, sigma_h4)
    -> scalar
```

For each of `n_groups` jackknife replicates, the package raises the `sigma_*` entries of one disjoint bin-group to

1e20 (masking them from the NNLS), re-solves the NNLS for the replicate, computes `chi2_func`, and finally returns `std(chi2_replicate)`. Replicates are streamed through `jax.lax.map` in batches of `batch_size` so peak memory stays bounded regardless of `n_groups`.

Critical: see pitfall §14.4 for how `chi2_func` must handle `sigma_xy` (masking + your own error model).

6.6 `likelihood.model_likelihood` — canonical MCMC log-likelihood

```
def model_likelihood(params, dict_data, num_Vbin, Rzphi_n_tot,
                    sigma_amplify=1.0) -> scalar
```

Takes the 12-D θ , calls `build_model`, computes per-bootstrap χ^2 against Σ , h_1 – h_4 (and $Rzphi$ density), does log-mean-exp over bootstraps $\rightarrow$ `logL_marg`. Use this to drive an MCMC. `sigma_amplify` multiplies the χ^2 denominator (data errors) if you want an inflation factor.

6.7 `likelihood.calculate_delta_chi2` — jackknife wrapper

```
def calculate_delta_chi2(params, dict_data, num_Vbin, Rzphi_n_tot) -> scalar
```

Thin wrapper: unpacks $\theta \rightarrow$ dicts, calls `model_deltaChi2_jackknife` with a default 100-group jackknife. Read the source at `likelihood.py:249` to see the exact `chi2_func` it uses if you need to customise.

6.8 Rotation curve / `L_circ` helpers

```
def get_rotation_curve(R, potential_fn, potential_args=(),
                      z=0.0, dR=1e-3):
    # v_c(R) = sqrt(R * dPhi/dR) at phi = 0
    # returns kpc/Gyr — multiply by KPCGYR_TO_KMS for km/s
def build_Lcirc_of_E(potential_fn, potential_args=(),
                    R_min=1e-2, R_max=1e2, n_grid=2000, dR=1e-3):
    # returns (Lcirc_interp: callable, E_grid, L_grid)
```

`get_rotation_curve` is JIT'd and `jax-transformable`. `build_Lcirc_of_E` returns a `scipy interp1d` (host-side) — use it to compute circularity $\lambda_z = L_z / L_{\text{circ}}(E)$.

7. Common workflows

7.1 Convert θ to halo/baryon dicts (mandatory)

Every entry point except the likelihood wrappers expects two dicts, not the raw vector. Standard helper:

```
def unpack_params(theta):
    """12-D vector (angles in RAD) -> halo/baryon dicts (angles in DEG)."""
    (logM_10kpc, logM_disk, logM_bar, logc_halo,
     logRs_disk, logHs_disk, logL_bar,
     alpha_rad, beta_rad, gamma_rad,
     log_LM, log_Omega) = [float(x) for x in theta]
    logM_halo, logRs_halo = logMenc_logc_to_logM_logRs(
        logM_10kpc, logc_halo, r_enc=10.0, Delta=200., rho_crit=277.54)
    L_bar = 10.0 ** logL_bar
    Hs = 10.0 ** logHs_disk
    params_halo_pot = dict(
        logM=float(logM_halo), Rs=10.0**float(logRs_halo),
        a=1.0, b=1.0, c=1.0,
        x_origin=0.0, y_origin=0.0, z_origin=0.0,
```

```

    dirx=0.0, diry=0.0, dirz=1.0,
)
params_baryon_rho = dict(
    logM_disc=logM_disk, Rs_disc=10.0**logRs_disk, Hs_disc=Hs,
    logM_bar=logM_bar, L_bar=L_bar, a_bar=L_bar/5.0, b_bar=Hs,
    mass_to_light_ratio=10.0**log_LM,
    Omega_bar=10.0**log_Omega,
    x_origin=0.0, y_origin=0.0, z_origin=0.0,
    dirx=0.0, diry=0.0, dirz=1.0,
    alpha=alpha_rad * 180/np.pi, # DEG
    beta =beta_rad * 180/np.pi,
    gamma=gamma_rad * 180/np.pi,
)
return params_halo_pot, params_baryon_rho

```

Copy this verbatim into new code. It's canonical.

7.2 Define density and potential closures (mandatory)

```

from potentials import (NFW_potential,
                        MiyamotoNagai_potential, MiyamotoNagai_density)
from dehnen_bar import (T3_potential, T3_density,
                        V4_potential, V4_density)

V4_A, V4_B, V4_L, V4_GAMMA = 0.5, 0.5, 0.1, 0.0
GAMMA_BAR = 1.0

@jax.jit
def density_func(x, y, z, params):
    mn_params = {k: params[k] for k in
                  ('logM_disc', 'Rs_disc', 'Hs_disc',
                   'x_origin', 'y_origin', 'z_origin',
                   'dirx', 'diry', 'dirz')}
    rho_mn = MiyamotoNagai_density(x, y, z, mn_params)
    M_bar = 10.0 ** params['logM_bar']
    rho_t3 = T3_density(x, y, z, M_bar, params['a_bar'],
                       params['b_bar'], params['L_bar'], GAMMA_BAR)
    rho_v4 = V4_density(x, y, z, M_bar, V4_A, V4_B, V4_L, V4_GAMMA)
    return rho_mn + rho_t3 + rho_v4

@jax.jit
def potential_func(x, y, z, params_baryon, params_halo):
    phi_halo = NFW_potential(x, y, z, params_halo)
    mn_params = {k: params_baryon[k] for k in
                  ('logM_disc', 'Rs_disc', 'Hs_disc',
                   'x_origin', 'y_origin', 'z_origin',
                   'dirx', 'diry', 'dirz')}
    phi_mn = MiyamotoNagai_potential(x, y, z, mn_params)
    M_bar = 10.0 ** params_baryon['logM_bar']
    phi_t3 = T3_potential(x, y, z, M_bar, params_baryon['a_bar'],
                         params_baryon['b_bar'], params_baryon['L_bar'],
                         GAMMA_BAR)
    phi_v4 = V4_potential(x, y, z, M_bar, V4_A, V4_B, V4_L, V4_GAMMA)
    return phi_halo + phi_mn + phi_t3 + phi_v4

```

Again — copy verbatim. `V4_A`, `V4_B`, `V4_L`, `V4_GAMMA`, `GAMMA_BAR` are "soul-of-the-model" constants; do not change without deep understanding.

7.3 Forward model at one θ (for a plot)

```

p_halo, p_baryon = unpack_params(theta)
model_dict = build_model(
    density_func, potential_func,
    p_halo, p_baryon, dict_data,
    num_Vbin=dict_data['total_bins'],
    Rzphi_n_tot=dict_data['Rzphi_n_tot'],
    Rzphi_n_grid=dict_data['Rzphi_n_grid'],
    Rzphi_lim_grid=jnp.array([dict_data['R_minmax'],
                              dict_data['z_minmax'],
                              dict_data['phi_minmax']]),
    xy_lim_grid=jnp.array([dict_data['X_minmax'],
                           dict_data['Y_minmax']]),
    xy_n_grid=jnp.array(dict_data['nX_nY']),
)
# Take row 0 (unperturbed) for plotting:
density_model = np.asarray(model_dict['surface_density'][0])
V_model       = np.asarray(model_dict['V'][0])
h1_model      = np.asarray(model_dict['h1'][0])
# ... etc

```

Model surface density is in **the same luminosity units** as `dict_data['XY_density_data']`. Compare directly.

7.4 Orbital library with trajectories

```

p_halo, p_baryon = unpack_params(theta_bestfit)
orb = get_model_with_orbit(density_func, potential_func,
                           p_halo, p_baryon, dict_data,
                           num_Vbin=dict_data['total_bins'], ...)

# Every accepted timestep = one weighted particle sample.
weights = np.asarray(orb['weights'])           # (n_particles,)
y_traj  = np.asarray(orb['y_traj'])           # (n_particles, 1, N_max, 6)
t_traj  = np.asarray(orb['t_traj'])
dt       = np.diff(t_traj[:, 0], axis=1, prepend=t_traj[:, 0, :1]) # (n_particles, N_max)
accepted = dt > 0
y_flat  = y_traj[:, 0][accepted]              # (N_accepted, 6)
w_flat  = (dt * weights[:, None])[accepted]   # (N_accepted,) = dt * orbit weight

# Apply 4-fold bar symmetry:
SIGN_SYM = np.array([
    [ 1, 1, 1, 1, 1, 1], [ 1, 1,-1, 1, 1,-1],
    [-1,-1, 1,-1,-1, 1], [-1,-1,-1,-1,-1,-1],
], dtype=np.float32)
y_sym = (y_flat[None] * SIGN_SYM[:, None, :]).reshape(-1, 6)
w_sym = np.tile(w_flat, 4)

```

`example_analysis.ipynb` Section 4 (in the example folder) shows the full pipeline down to a circularity histogram.

7.5 MCMC driver skeleton

Not fully unrolled here — see `example/example_schwarmax.ipynb` for the canonical BlackJAX adaptive-RMH driver. The critical thing is that the MCMC calls `model_likelihood` (or a very close variant of it) and does **not** call `build_model` directly. The likelihood wrapper does the θ -unpacking; the MCMC just sees a scalar function of θ .

7.6 Jackknife error bar

```

def chi2_func(y_Rzphi, y_Rzphi_model, sigma_Rzphi,
             y_xy, y_xy_model, sigma_xy,          # in luminosity units

```



```

        y_h1,    y_h1_model,    sigma_h1,
        y_h2,    y_h2_model,    sigma_h2,
        y_h3,    y_h3_model,    sigma_h3,
        y_h4,    y_h4_model,    sigma_h4):
# Respect the mask: replicates raise sigma_xy to 1e20 on dropped bins.
# Override to 1% relative error ONLY on kept bins:
sig_xy = jnp.where(sigma_xy < 1e10, 0.01 * y_xy, sigma_xy)
res_xy = jnp.nansum(((y_xy - y_xy_model) / sig_xy) ** 2)
res_h1 = jnp.nansum(((y_h1 - y_h1_model) / sigma_h1) ** 2)
# ... etc for h2, h3, h4
return -0.5 * (res_xy + res_h1 + res_h2 + res_h3 + res_h4)

delta_chi2 = model_deltaChi2_jackknife(
    density_func, potential_func, chi2_func,
    p_halo, p_baryon, dict_data,
    num_Vbin=dict_data['total_bins'], ...,
    n_groups=100, batch_size=25)

```

8. Building your own forward model

Sections 6–7 covered the **high-level** entry points (`build_model`, `get_model_with_orbit`, `model_likelihood`, `calculate_delta_chi2`). Those are enough for 95% of use cases: run an MCMC, evaluate the model at a best-fit θ , make a plot.

For the remaining 5% — building a *different* forward model (a new potential, a different orbit initialisation, a new NNLS constraint set, a different binning scheme, etc.) — you assemble the pipeline yourself from the low-level pieces. The pieces are:

<pre>density_func(x, y, z) potential_func(x, y, z)</pre>	1. Draw stellar positions 2. Solve Jeans for (v_{rot}, σ_{gas}) 3. Turn $(x, y, z) \rightarrow (x, y, z, v_x, v_y, v_z)$ 4. Integrate each orbit for N_{orb} periods in the rotating frame 5. Bin each orbit's positions into Voronoi & R_{zphi} grids; compute Gauss-Hermite moments 6. Build the design matrix A and solve NNLS for weights w 7. Forward-model outputs are $A @ w$
--	---

Each step is a public function. The high-level `build_model` is literally the composition of these seven pieces (read `model.py:71` end-to-end to see it). Anything you'd want to customise (say, use a different potential in step 5 while keeping steps 1–3 unchanged) is a matter of assembling the pieces differently.

The next five sections walk through each piece with signatures, canonical arguments, return-shape conventions, and traps.

9. Initial conditions

Two-step process: (a) draw (x, y, z) positions from an analytic-disc PDF, (b) turn positions into velocities via a Jeans integral.

9.1 Position sampler

`load_data_bootstrap` already returns `dict_data['w0']` with shape `(n_samples, 3)` — the default sampler uses:

- R drawn from an $X \cdot \exp(X)$ distribution with scale 3 kpc (this is the classic exponential-disc surface-density profile).
- ϕ uniform in $[0, 2\pi)$.
- $|z|$ drawn from $\exp(X)$ with scale 1.2 kpc.

If you want a different position sampler, replace the `w0` field of the dict *after* `load_data_bootstrap` returns:

```
from utils import XexpX_pdf_log, expX_pdf_log, sample_from_logP

x_grid = np.linspace(0., 20., 2000)
R_samples = sample_from_logP(x_grid, XexpX_pdf_log(x_grid, R_s=5.0),
                             n=10_000, key=jax.random.PRNGKey(42))
z_samples = sample_from_logP(x_grid, expX_pdf_log(x_grid, H_s=0.5),
                             n=10_000, key=jax.random.PRNGKey(43))
phi_samples = np.random.default_rng(42).uniform(0, 2*np.pi, size=10_000)
dict_data['w0'] = np.stack([R_samples * np.cos(phi_samples),
                           R_samples * np.sin(phi_samples),
                           z_samples], axis=1)
```

`sample_from_logP` is inverse-transform sampling of a log-PDF grid. $XexpX_pdf_log(x, R_s) = \log(x \cdot \exp(-x/R_s) / R_s^2)$, the exponential-disc surface density. $expX_pdf_log(x, H_s) = \log(\exp(-x/H_s) / H_s)$, the vertical exponential profile. All three helpers are in `utils.py`.

9.2 Jeans-derived velocities

The `jeans.py` module gives each position a 3-D velocity from the axisymmetric Jeans equations under the assumption $\sigma_R = \sigma_\phi = \sqrt{\text{anisotropy_b}} \cdot \sigma_z$.

```
from jeans import get_jeans_moments, get_w0_new

# Per-particle four numbers:
v_rot, sigma_R, sigma_z, sigma_phi = get_jeans_moments(
    x_star, y_star, z_star,
    params_baryon, params_halo,
    potential_func, density_func,
    anisotropy_b=1.0)

# Vectorised: turn a whole (n, 3) position array into (n, 6) phase-space:
w0_full = get_w0_new(w0, key1, key2, key3, n,
                     params_baryon, params_halo,
                     potential_func, density_func)

# w0_full[:, :3] = positions (unchanged)
# w0_full[:, 3:] = (vx, vy, vz) drawn from  $N(v_{\text{rot}} \cdot \hat{e}_\phi, \text{diag}(\sigma_R, \sigma_z, \sigma_\phi))$ 
```

`get_w0_new` is what `build_model` calls internally at step 3. Use it directly if you want to inspect Jeans moments before integration or to inject your own velocity draws.

Signatures:

```
def get_jeans_moments(x_star, y_star, z_star,
                     params_baryon, params_halo,
                     potential_func, density_func,
                     anisotropy_b=1.0):
    # -> (v_mean_phi, sigma_R, sigma_z, sigma_phi)    scalar 4-tuple

def get_w0_new(w0, key1, key2, key3, n_particles,
```

```

        params_baryon, params_halo,
        potential_func, density_func):
    # w0: (n_particles, 3) positions -> (n_particles, 6)

```

9.3 Multiple realisations per particle

`build_model` uses `n_realizations = 4` — every particle is duplicated into 4 copies with small uniform jitter added to the position (± 0.05 kpc) and velocity ($\pm 0.05 \cdot v_c$, clipped to $[1, 15]$ km/s). The four copies are integrated independently and their orbital-library contributions are stacked into the design matrix. This averages over the intrinsic scatter of Jeans-drawn velocities.

`get_model_with_orbit` uses `n_realizations = 1` to keep the trajectory tensor small (one 6-D trajectory per particle instead of four).

If you're rolling your own, copy the pattern from `model.py:127–141`:

```

n_realizations = 4
keys = jax.random.split(jax.random.PRNGKey(911), 6)
d_scale = 0.1 * jnp.ones(_R.shape)          # 0.1 kpc position
v_scale = jnp.clip(0.1 * _Vc, 1, 15)         # 10% of local v_c, clipped
noise = jnp.stack([
    (jax.random.uniform(keys[i], (n, n_realizations)) - 0.5) *
    (d_scale if i < 3 else v_scale)[: , None]
    for i in range(6)
], axis=-1)                                # (n, n_realizations, 6)
w0_batch = w0_new[:, None, :] + noise

```

10. Orbit integration

The integrator is an **adaptive Bogacki–Shampine BS(2,3)** leapfrog in the bar's rotating frame at pattern speed Ω_{bar} . Absolute / relative tolerances default to $1e-7$ / $1e-4$. Step-size floor `dt_min = 1e-5` Gyr, ceiling `dt_max = 0.3` Gyr.

Four public functions, all in `integrates.py`:

function	trajectory stored?	vectorises over orbits?
<code>integrate_adaptive_barred_chunked</code>	no (streamed, chunk-wise)	no (single orbit)
<code>integrate_adaptive_batch_chunked</code>	no	yes (batch axis 0)
<code>integrate_adaptive_barred_withtraj</code>	yes (full $N_{\text{max}} \times 6$)	no
<code>integrate_adaptive_withtraj_batch</code>	yes	yes

Rule of thumb:

- **Building an orbital library** (Steps 4–5 of the pipeline) → use the chunked variants. Memory is $O(\text{chunk_size} \cdot 6)$ per orbit regardless of N_{max} .
- **Post-analysis** where you need to plot individual orbits or compute per-timestep quantities (energies, circularities, angular momentum evolution) → use the `withtraj` variants and pay the $O(N_{\text{max}} \cdot 6)$ memory cost per orbit.

10.1 Signatures (canonical arguments only)

```

def integrate_adaptive_batch_chunked(
    w0,                                # (n_orb, 6) initial (x,y,z,vx,vy,vz)

```

```

    acc_fn, pot_fn,                # closures over the potential (see $11)
    N_max,                        # max integration steps (static)
    T_total,                      # (n_orb,) per-orbit total integration time
    dt_init=0.010, Omega=0.0,
    atol=1e-8, rtol=1e-6,
    dt_min=1e-5, dt_max=0.1,
    # -- binning grid parameters --
    num_Vbin=1028,
    bin_mapping=jnp.zeros(2400, dtype=jnp.int32),
    num_per_bin=jnp.zeros(1028, dtype=jnp.int32),
    Rzphi_minmax=jnp.array([[0,10],[-3,3],[-pi,pi]]),
    XY_minmax=jnp.array([[ -10,10],[-2,2]]),
    nRzphi=jnp.array([10,6,6]), nXY=jnp.array([40,30]),
    num_segments_Rzphi=360,
    # -- Gauss-Hermite basis --
    v0=jnp.zeros(1028), s=jnp.ones(1028)*5.0,
    rotation_matrix=jnp.eye(3),
    chunk_size=500,
):
    # Returns: (Rzphi_bin_counts_out, surface_density_out, h0..h4_out,
    #          valid_count).
    # Each *_out is (n_bins,) – the ORBITAL LIBRARY averaged over the
    # n_orb input orbits. To recover per-orbit design-matrix rows, use
    # the vmap variant `_integrate_adaptive_batch_chunked_vmap` (below).

```

The vmap wrapper (used by `build_model`):

```

from integrates import _integrate_adaptive_batch_chunked_vmap

Rzphi_bin_counts, surface_density, h0, h1, h2, h3, h4, valid = \
    _integrate_adaptive_batch_chunked_vmap(
        w0_batch,                # (n_particles, n_realizations, 6)
        acc_fn, pot_fn,
        N_max, T_total_batch, # T_total_batch shape (n_particles, n_realizations)
        dt_init_batch, -Omega_bar,
        atol, rtol, dt_min, dt_max,
        num_Vbin, bin_mapping, num_per_bin,
        Rzphi_lim_grid, xy_lim_grid,
        Rzphi_n_grid, xy_n_grid, Rzphi_n_tot,
        v0, s, rotation_matrix,
        chunk_size,             # e.g. 100
    )
# Each output is (n_particles, n_bins). Transpose for the NNLS
# design matrix: A_Rzphi = Rzphi_bin_counts.T → (n_bins, n_orbits).

```

The `withtraj_batch` variant returns two extra tensors:

```

_, _, _, _, _, _, valid, y_traj, t_traj = \
    _integrate_adaptive_withtraj_batch_vmap(w0_batch, acc_fn, pot_fn,
        N_max, T_total_batch, ...)
# y_traj : (n_particles, n_realizations, N_max, 6)  positions + velocities
# t_traj : (n_particles, n_realizations, N_max)      integration time (Gyr)
# valid   : scalar per orbit – number of accepted steps

```

10.2 Time-scale and step-count conventions

`build_model` fixes:

- `N_dynamical_time = 50` — integrate for 50 orbital periods.
- `N_step_per_orb = 100` — target ~100 steps per period → `N_max = 5000`.

- `T_total_batch = T_orb · 50` — per-particle total time.
- `dt_init_batch = T_orb / 100.`

`T_orb` comes from `utils.estimate_orbital_timescale(R, potential_fn, args, z)`, which computes $T = 2\pi / \Omega(R, z)$ where $\Omega^2(R, z) = (1/R) d\Phi/dR + d^2\Phi/dz^2$ (epicyclic approximation).

If your potential has very different dynamical times you may need larger `N_max`. Note that `N_max` must be a **static** argument, so JAX recompiles when you change it.

10.3 Chunked binning — how the design matrix is built

The integrator bins each accepted timestep into two grids as it goes:

- **Rzphi_bin_counts** — a 3-D histogram in (R, z, ϕ) of the orbit's positions, weighted by `dt`. Shape `(n_Rzphi_bins,)` per orbit.
- **surface_density** (aka the XY design row) — a 2-D histogram in $(X_{\text{sky}}, Y_{\text{sky}})$ using the rotation matrix from viewing angles. Voronoi bin index is looked up via `bin_mapping` on the pixel grid. Shape `(num_Vbin,)` per orbit.

Also **on-the-fly** at each accepted timestep, the integrator computes per-orbit un-normalised Gauss-Hermite moments:

```
sw_k = Σ_t dt · GH_k( (v_los(t) - v0_bin) / s_bin )      for k = 0..4
```

The `A_h_k` outputs are `sw_k / T_integrated` — un-normalised (Agama's "option-b" convention). They divide by `gamma_kin` at the NNLS stage, not here.

10.4 Static args and JAX compilation

The following are `static_argnames`:

- `acc_fn, pot_fn` — closures; hash by identity, so build once and reuse.
- `N_max, chunk_size, num_Vbin, num_segments_Rzphi` — integers.

Everything else is a runtime array. If you sweep parameter vectors, you only pay the compilation cost once *per* structural change (e.g. changing `num_Vbin` because you loaded a different `dict_data`).

11. Specifying the potential

Two supported patterns.

11.1 Analytic potential–density pairs (default)

The pattern used everywhere in `example_analysis.ipynb`:

```
@jax.jit
def potential_func(x, y, z, params_baryon, params_halo):
    return (NFW_potential(x, y, z, params_halo)
            + MiyamotoNagai_potential(x, y, z, mn_params(params_baryon))
            + T3_potential(x, y, z, M_bar,
                           params_baryon['a_bar'],
                           params_baryon['b_bar'],
                           params_baryon['L_bar'], GAMMA_BAR)
            + V4_potential(x, y, z, M_bar, V4_A, V4_B, V4_L, V4_GAMMA))
```

Available potentials and their canonical parameter dicts:

function	in file	parameter dict keys
NFW_potential(x, y, z, params)	potentials.py:19	logM, Rs, a, b, c, x_origin, y_origin, z_origin, dirx, diry, dirz
MiyamotoNagai_potential(x, y, z, params)	potentials.py:52	logM_disc, Rs_disc, Hs_disc, x_origin, y_origin, z_origin, dirx, diry, dirz
T3_potential(x, y, z, M, a, b, L, γ)	dehnen_bar.py:1562	positional args
T4_potential(...)	dehnen_bar.py:1563	positional args
V4_potential(...)	dehnen_bar.py:1568	positional args

Each has a matching `*_density(...)` in the same file with an identical signature (the T/V families are automatically generated factory pairs at `dehnen_bar.py:1559-1568`).

`x_origin, y_origin, z_origin` shift the component's centre. `dirx, diry, dirz` are the components of the density's symmetry axis (unit vector). For a flat disc leave these as `0, 0, 1`.

11.2 Custom analytic potential

To add your own analytic potential, define **matching** (`density(x,y,z,params), potential(x,y,z,params)`) closures with the same interface as the built-in ones and drop them into `density_func / potential_func`. The integrator only needs `potential_func` to compute acceleration via `jax.grad`; the `density_func` is only used inside the Jeans-moment integrator and inside the 3-D density constraint (step 5 of the pipeline).

The `acc_fn` and `pot_fn` closures that `integrate_adaptive_*` consumes are the standard wrap:

```
@jax.jit
def acc_fn(x, y, z):
    def _pot(pos):
        return potential_func(pos[0], pos[1], pos[2], p_baryon, p_halo)
    return -jax.grad(_pot)(jnp.array([x, y, z]))

@jax.jit
def pot_fn(x, y, z):
    return potential_func(x, y, z, p_baryon, p_halo)
```

11.3 Cylindrical-spline representation

For potentials that don't have a closed-form expression (e.g. an N-body density model, an arbitrary triaxial bar) use the cylindrical-spline expansion in `CylindricalSpline.py`.

Signature and workflow:

```
from CylindricalSpline import get_phi_m, evaluate_phi, get_acc

# 1. Build the spline. rho_fn(x, y, z, params) is any callable.
# Static args below → JAX recompiles when they change.
phi_dict = get_phi_m(
    rho_fn, params,
    NR=50, NZ=30,          # spline grid points in R and z
    Rmin=1e-3, Rmax=30.,   # radial extent (kpc)
    Zmin=1e-3, Zmax=15.,   # vertical extent (kpc)
    Mmax=8,                # Fourier m modes retained
    Nphi=24,               # azimuthal samples per rho_m integral
```

```

    N_int=1600,          # → base = sqrt(N_int) rounded up-to-odd Simpson pts
)
# phi_dict has keys: 'Rgrid', 'Zgrid', 'R0', 'm', 'Phi_m_real', 'Phi_m_img',
#                   'Mx_real', 'My_real', 'Mx_img', 'My_img'

# 2. Evaluate the potential (scalar-in, scalar-out; use jax.vmap for arrays):
phi_val = evaluate_phi(x, y, z, phi_dict)

# 3. Wrap it into schwarmax's potential_func signature:
@jax.jit
def potential_func(x, y, z, params_baryon, params_halo):
    return evaluate_phi(x, y, z, params_baryon['phi_dict'])

```

Best-practice hyperparameters (validated via `SchwarMAX/benchmark_cylspline.py` at the sibling repo — the tests covered MN discs, T3 bars, and AGAMA cross-checks):

- **Rmax, Zmax** must comfortably enclose the source density. Too tight → the Green's-function integral truncates the mass tail and $|\Phi|$ is systematically underestimated. For a Miyamoto-Nagai disc with $R_s = 3$ kpc, use $R_{\max} \geq 50$, $Z_{\max} \geq 20$ for < 1% median error.
- **NR, NZ** = 25–50 each. Beyond that you saturate the integration resolution instead of the spline grid.
- **Mmax** = 4 (T3 bar's dominant modes are $m = 0, 2, 4$). Higher is harmless but doubles the cost.
- **Nphi** $\geq 2 \cdot M_{\max} + 1$ (Nyquist). 24 is plenty for $M_{\max} = 4$.
- **N_int** = 1600 (→ 41×41 Simpson points). Beyond this, gains vanish quickly.

For the T3 bar (compact, $R_{\max} = 20$ fine) or the V4 bulge (compact), the defaults $R_{\max} = 30$, $Z_{\max} = 15$ in `example/` integrate cleanly. For the MN disc the defaults truncate; if you CylSpline-model the *whole* baryon system use $R_{\max} = 50$.

Analytic derivatives are available:

```

acc = get_acc(x, y, z, phi_dict)      #  $-\nabla\Phi$  (vector, kpc/Gyr2)
H   = get_hessian(x, y, z, phi_dict)  #  $\partial^2\Phi/\partial x_i \partial x_j$ 
rho = get_density(x, y, z, phi_dict)  # Poisson-verified density

```

`get_acc` uses **analytic** derivatives of the cubic-spline interpolant — cheaper and more accurate than `jax.grad` through `evaluate_phi`. Prefer it in the integrator's `acc_fn`.

11.4 Mixed: analytic disc + CylSpline bar

Typical setup for a bar galaxy where the disc has a closed form but the bar is data-driven:

```

# Build CylSpline for the bar once, at load time:
bar_phi_dict = get_phi_m(bar_density_fn, bar_params,
                        NR=40, NZ=30, Rmin=1e-3, Rmax=20.,
                        Zmin=1e-3, Zmax=10., Mmax=4,
                        Nphi=16, N_int=1600)

@jax.jit
def potential_func(x, y, z, p_baryon, p_halo):
    return (NFW_potential(x, y, z, p_halo)
            + MiyamotoNagai_potential(x, y, z, mn_params(p_baryon))
            + evaluate_phi(x, y, z, bar_phi_dict))

```

The disc mass and viewing angles remain free MCMC parameters; the bar shape is frozen by the CylSpline (a good approximation if the bar density is well-constrained externally).

12. Using the NNLS solver directly

The solver assembles a stacked linear system from six data channels (Rzphi density, XY surface density, and h_1 – h_4) and finds non-negative orbital weights that minimise a Tikhonov-regularised residual.

12.1 Design-matrix construction

The stacked "U matrix" and RHS "y" are built inside `solve_nnls_admm` from the six per-orbit design-matrix blocks `A_Rzphi`, `A_xy`, `A_h1..A_h4`:

```
U = w_rzphi · A_Rzphi / σ_Rzphi          # 3D density rows
    w_xy    · A_xy    / σ_xy              # 2D surface-density rows
    w_h     · A_hk / (γ_kin · σ_hk)      k=1..4 # kinematic rows

y = w_rzphi · y_Rzphi / σ_Rzphi
    w_xy    · y_xy    / σ_xy
    w_h     · y_hk    / σ_hk              k=1..4
```

$\gamma_{kin_b} = M_b / (norm_b \cdot 2 \cdot \sqrt{\pi} \cdot s_b)$ is the data's h_0 in the per-bin basis (v_0, s); it converts the un-normalised model h-moments (which come out of the integrator as sw_k/T) into the same units as the data h-moments.

Then

```
Q = UTU + (λreg / norb) · I
c = -UT y
```

and $NNLS = \min \frac{1}{2}w^T Q w + c^T w$ s.t. $w \geq 0$.

12.2 Single-observation solver

```
from nnls import solve_nnls_admm

w = solve_nnls_admm(
    A_Rzphi, A_xy, A_h1, A_h2, A_h3, A_h4,      # (n_bins, n_orb) each
    y_Rzphi, y_xy, gamma_kin,                  # (n_bins,)
    y_h1, y_h2, y_h3, y_h4,                    # (n_bins,)
    sig_Rzphi, sig_xy, sig_A1, sig_A2, sig_A3, sig_A4, # (n_bins,)
    lambda_reg=1.0, maxiter=200,
    w_rzphi=jnp.sqrt(5.0), w_xy=jnp.sqrt(5.0), w_h=jnp.sqrt(1.0),
)
# w : (n_orb,), all ≥ 0
```

Under the hood: shared Cholesky factorisation of $Q + \rho I$ (where $\rho = \text{trace}(Q) / n_{orb}$), then a `jax.lax.scanned` ADMM iteration. `maxiter = 200` is enough for $n_{orb} \sim 5000$, $n_{bins} \sim 600$. `lambda_reg` = ridge penalty on $w^T w$.

`w_rzphi`, `w_xy`, `w_h` are **channel weights** (sqrt of relative scaling — they enter the residual as `w_channel · residual`). The defaults inside `build_model` are `w_rzphi = w_xy = √5`, `w_h = √1`, i.e. the density constraints are weighted 5× harder than the kinematics; this makes the solver preserve the surface-density fit while allowing kinematic slack.

12.3 Bootstrap solver — shared Cholesky, vmapped RHS

For a bootstrap-marginalised likelihood you want `Nboot` NNLS solves against the same design matrix but perturbed observations. The bootstrap solver factors Q once and vmaps only the ADMM iteration:

```
from nnls import solve_nnls_admm_bootstrap
```



```
w_all = solve_nnls_admm_bootstrap(
    A_Rzphi, A_xy, A_h1, A_h2, A_h3, A_h4,
    y_Rzphi, y_xy, gamma_kin,                # shared (unperturbed) refs
    y_xy_boot, y_h1_boot, y_h2_boot, y_h3_boot, y_h4_boot, # (N_boot, n_bins)
    sig_Rzphi, sig_xy, sig_A1, sig_A2, sig_A3, sig_A4,
    lambda_reg=1.0, maxiter=200,
)
# w_all : (N_boot, n_orb)
```

Row 0 is unperturbed (since `dict_data['*_standard_normal'][0]` is zeroed in `load_data_bootstrap`). Rows 1..`N_boot`-1 are perturbed. This is exactly what `build_model` calls internally.

`y_Rzphi` and `sig_Rzphi` are **not** bootstrapped — the 3-D density constraint is a model prediction, not an observation.

12.4 Computing model outputs from the weights

`compute_model_single(w, A_Rzphi, A_xy, A_h1..A_h4, gamma_kin, v0, s)` in `model.py:15` takes the weights and produces the eight model maps:

```
from model import compute_model_single

density_3d, density_2DXY, h1, h2, h3, h4, V, sigma = compute_model_single(
    w, A_Rzphi, A_xy, A_h1, A_h2, A_h3, A_h4, gamma_kin, v0, s,
)

• density_3d = A_Rzphi @ w
• density_2DXY = A_xy @ w
• h_k = (A_hk @ w) / v_kin, clipped to  $\pm 10$ 
• (V,  $\sigma$ ) = h_to_V_sigma(h1, h2, v0, s, h3, h4) — vdMF conversion in ghmoments.py:54
```

For a full posterior sweep use the vmapped version at `model.py:68`: `compute_model_single_vmap`.

13. End-to-end DIY forward model

Putting everything together — this is what `build_model` does internally, written out as a script you can adapt:

```
import jax, jax.numpy as jnp, numpy as np
from utils import (load_data_bootstrap, get_rotation_curve,
                   estimate_orbital_timescale, makeRotationMatrix)
from jeans import get_w0_new
from integrates import _integrate_adaptive_batch_chunked_vmap
from nnls import solve_nnls_admm
from model import compute_model_single
from constants import EPSILON

# 1. Data + IC positions
dict_data = load_data_bootstrap('/path/to/', 'mock.pkl', n_samples=5000)
w0_pos = dict_data['w0'] # (n_particles, 3)
n = w0_pos.shape[0]

# 2. Assemble your (density, potential) closures (see §11).
#    p_baryon, p_halo built from your  $\theta$  (see §7.1).

# 3. Jeans-driven initial velocities
key1, key2, key3 = (jax.random.PRNGKey(k) for k in (42, 109, 2026))
w0_full = get_w0_new(w0_pos, key1, key2, key3, n,
```

```

        p_baryon, p_halo, potential_func, density_func)

# 4. Jittered realisations (see §9.3)
_R = jnp.sqrt(w0_full[:, 0]**2 + w0_full[:, 1]**2)
_Vc = jax.vmap(get_rotation_curve, in_axes=(0, None, None, 0))(
    _R, potential_func, (p_baryon, p_halo), w0_full[:, 2])

n_realizations = 4
keys = jax.random.split(jax.random.PRNGKey(911), 6)
d_scale = 0.1 * jnp.ones(_R.shape)
v_scale = jnp.clip(0.1 * _Vc, 1, 15)
noise = jnp.stack([
    (jax.random.uniform(keys[i], (n, n_realizations)) - 0.5) *
    (d_scale if i < 3 else v_scale)[: , None]
    for i in range(6)
], axis=-1)
w0_batch = w0_full[:, None, :] + noise

# 5. Orbit integration + on-the-fly binning
T_orb = jax.vmap(estimate_orbital_timescale, in_axes=(0, None, None, 0))(
    _R, potential_func, (p_baryon, p_halo), w0_full[:, 2])
T_orb_batch = T_orb[:, None].repeat(n_realizations, axis=1)
N_step_per_orb, N_dynamical_time = 100, 50
N_max = N_step_per_orb * N_dynamical_time

@jax.jit
def acc_fn(x, y, z):
    def _pot(pos): return potential_func(pos[0], pos[1], pos[2], p_baryon, p_halo)
    return -jax.grad(_pot)(jnp.array([x, y, z]))

@jax.jit
def pot_fn(x, y, z):
    return potential_func(x, y, z, p_baryon, p_halo)

rotation_matrix = makeRotationMatrix(alpha_deg, beta_deg, gamma_deg)
Omega_bar = 10 ** log_Omega      # kpc/Gyr

Rzphi_bin, xy_dens, h0, h1, h2, h3, h4, _ = _integrate_adaptive_batch_chunked_vmap(
    w0_batch, acc_fn, pot_fn, N_max,
    T_orb_batch * N_dynamical_time,
    T_orb_batch / N_step_per_orb, -Omega_bar,
    1e-7, 1e-4, 1e-5, 0.3,
    dict_data['total_bins'], dict_data['bin_mapping'], dict_data['num_per_bin'],
    jnp.array([dict_data['R_minmax'], dict_data['z_minmax'], dict_data['phi_minmax']]),
    jnp.array([dict_data['X_minmax'], dict_data['Y_minmax']]),
    dict_data['Rzphi_n_grid'], jnp.array(dict_data['nX_nY']),
    dict_data['Rzphi_n_tot'],
    dict_data['v0'], dict_data['s'], rotation_matrix,
    100, # chunk_size
)
A_Rzphi = Rzphi_bin.T      # (n_bins, n_particles)
A_xy     = xy_dens.T
A_h1, A_h2, A_h3, A_h4 = h1.T, h2.T, h3.T, h4.T

# 6. Data + errors + normalisation
y_xy     = dict_data['XY_density_data'] * (10 ** log_LM)      # to mass units
sig_xy   = dict_data['XY_density_data_err'] * (10 ** log_LM)
y_hks    = [dict_data[f'h{k}_data'] for k in (1, 2, 3, 4)]
sig_hks  = [dict_data[f'h{k}_data_err'] + EPSILON for k in (1, 2, 3, 4)]
y_Rzphi, sig_Rzphi = ... # from your density integral (see model.py:207-234)

M_per_bin = y_xy * dict_data['num_per_bin'] * (
    (dict_data['X_minmax'][1] - dict_data['X_minmax'][0]) / dict_data['nX_nY'][0] *
    (dict_data['Y_minmax'][1] - dict_data['Y_minmax'][0]) / dict_data['nX_nY'][1]) * 1e6

```

```

gamma_kin = M_per_bin / ((1 + dict_data['h4_data'] * jnp.sqrt(6.)/4) *
                        2 * jnp.sqrt(jnp.pi) * dict_data['s'] + EPSILON)

# 7. Solve NNLS
w = solve_nnls_admm(
    A_Rzphi, A_xy, A_h1, A_h2, A_h3, A_h4,
    y_Rzphi, y_xy, gamma_kin, *y_hks,
    sig_Rzphi, sig_xy, *sig_hks,
    lambda_reg=1.0, maxiter=200,
)

# 8. Model outputs
d3d, d2d, h1m, h2m, h3m, h4m, V_m, sigma_m = compute_model_single(
    w, A_Rzphi, A_xy, A_h1, A_h2, A_h3, A_h4, gamma_kin,
    dict_data['v0'], dict_data['s'],
)

```

Compare to `model.py:71-286` — the two are equivalent, up to `build_model` also handling bootstrap perturbations, unit conversions between mass/luminosity, and the mean-mass-per-orbit rescaling.

Rule of thumb: use `build_model` unless you specifically need to swap out a step. If you're swapping out step 4 (integrator) or step 6 (NNLS solve) you almost certainly want to keep the surrounding scaffolding.

14. Pitfalls — READ THIS

14.1 The `log_mass_to_light` sign convention

This is the single biggest footgun.

The **older** `SchwarMAX/` codebase defines slot 10 of θ as $\log(L/M)$ and pipes `light_to_mass_ratio = 10**theta[10]` into the forward model.

The **newer** `schwarmax_pkg/` (this package) defines slot 10 of θ as $\log(M/L)$ and pipes `mass_to_light_ratio = 10**theta[10]`. Inside `build_model` it inverts internally.

Same numerical value in slot 10 → **opposite physical meaning** in the two pipelines.

Consequence for old checkpoints: 13-D chains produced by the old `likelihoods_bar.py` need `theta[10] → -theta[10]` before feeding them to `build_model`. 12-D chains produced by `likelihood.model_likelihood` are already in the correct convention and do **not** need flipping.

`example_analysis.ipynb` auto-detects by `ndim` (13 → flip, 12 → don't). If you are writing new code that consumes an existing checkpoint: **check the shape** and flip accordingly.

14.2 Angles are radians in θ , degrees in the baryon dict

`build_model` reads `params_baryon['alpha']` in **degrees** and calls `makeRotationMatrix(alpha, beta, gamma)` which expects degrees. But the MCMC chain stores angles in **radians**. `unpack_params` (§7.1) converts. If you build the baryon dict by hand, remember to multiply by `180/np.pi`.

14.3 `build_model` returns bootstrap-stacked arrays

Every kinematic/density field is `(N_BOOTSTRAP, n_bins)`. Row 0 is the unperturbed data (`XY_standard_normal[0]` is zeroed in `load_data_bootstrap`), so `model_dict['surface_density'][0]` is your best-fit model for plotting.

14.4 Jackknife `chi2_func` — units and masking

Units: `model_deltaChi2_jackknife` passes `y_xy` and `sigma_xy` to your `chi2_func` in **luminosity units** (fixed in commit 5f59b9f onwards). Before that fix, `y_xy` was in internal NNLS units while `y_xy_model` was in luminosity — the resulting χ^2 was garbage.

Masking: the package's per-replicate masking works by raising `sigma_xy` to $1e20$ on dropped bins. If your `chi2_func` **overrides** `sigma` unconditionally (e.g. `sig_xy = 0.01 * y_xy`), the mask is lost and dropped bins are counted with a tiny error, inflating the jackknife std. The correct pattern:

```
sig_xy = jnp.where(sigma_xy < 1e10, 0.01 * y_xy, sigma_xy)
```

14.5 Kinematic bins are already un-normalised via `gamma_kin`

Model h-moments are computed as $(A_{hk} @ w) / \gamma_{kin}$ where $\gamma_{kin} = M_{per_bin} / (norm \cdot 2\sqrt{\pi} \cdot s)$ (Agama's option-b convention). The data h-moments are `dict_data['h1_data']` etc., not multiplied by anything. **Do not** divide the model h-moments by the surface density; that would double-normalise.

14.6 JIT static args and recompilation

The following args of `build_model` are static, meaning JAX recompiles when they change:

- `density_func, potential_func` (structural — normally frozen at module load)
- `num_Vbin, Rzphi_n_tot, nnls_maxiter`

Changing `n_samples` in `load_data_bootstrap` changes the number of particles $\rightarrow$ different `w0.shape[0]` $\rightarrow$ triggers a recompile. If you sweep many parameter vectors, keep `dict_data` fixed.

Also: don't pass Python floats where jax expects arrays (or vice versa). `params_halo_pot['logM']` must be a **Python float**, not a `jnp.float32`, to avoid re-tracing per posterior sample.

14.7 The N-body snapshot is 126 MB and Git-LFS-tracked

`example/Bar_model_TG21/model/t_t0_7` is stored in Git LFS. After cloning:

```
git lfs install      # one-time setup
git lfs pull         # actually download the snapshot
```

If you skip these, the file will be a 134-byte pointer and any snapshot-using cell will fail with an obscure error.

14.8 The `example_schwarmax.ipynb` cell 1 assumes Google Colab

The first cell does `from google.colab import drive` and pip-installs `jaxopt / corner / emcee / blackjax`. If you're running the notebook locally, replace this cell with a `sys.path.insert(0, "...")` and skip the pip installs (or run them via `pip install` manually).

14.9 GPU is highly recommended for MCMC

`build_model` on CPU takes ~60–100 s per call. For 16 chains $\times$ 3000 steps that's ~20 days of CPU time. On an A100 the same run is ~4 h. For **analysis** (single- θ or small posterior sweeps) CPU is fine.

15. The example folder

The example folder is the ground truth for how to use the package.

- `example_schwarmax.ipynb` — Colab-friendly BlackJAX adaptive RMH driver. Loads

`mock_Nbody_galaxy_beta65_gamma45_D50.pkl` via `load_data_bootstrap`, defines `density_func` + `potential_func` + `model_likelihood`, runs the MCMC, checkpoints every 5 steps.

- `example_analysis.ipynb` — 4-section post-MCMC tutorial:

1. Corner plot (posterior visualisation).
2. Data vs model in the disc-aligned frame (uses `build_model`).
3. Rotation curve + DM enclosed mass recovery (uses `get_rotation_curve` and analytic NFW; compares to N-body ground truth via AGAMA).
4. Orbital structure — circularity histogram from `get_model_with_orbit` vs N-body.

- `mcmc_checkpoint.pkl` — 12-D chain, 1000 steps × 32 chains, produced by `example_schwarmax.ipynb`.

- `mock_Nbody_galaxy_beta65_gamma45_D50.pkl` — Voronoi-binned mock IFU data (Σ , V , σ , h_1 – h_4 + errors + orientation) at $\alpha=25^\circ$, $\beta=65^\circ$, $\gamma=45^\circ$, distance 50 Mpc.

- `Bar_model_TG21/model/t_t0_7` — raw N-body snapshot (126 MB, Git LFS). Used for ground-truth density contours, halo $M(<r)$, and circularity.

- `Bar_model_TG21/model/t_t0_7.ini` — pre-computed AGAMA CylSpline potential from the snapshot. Loading is fast; use this for `v_c(R)` and `L_circ(E)` on the N-body side.

16. Where to look for canonical usage of each function

Function	Canonical use
<code>load_data_bootstrap</code>	<code>example_analysis.ipynb</code> Section 0, cell 5
<code>build_model</code>	<code>example_analysis.ipynb</code> Section 2, cell 11
<code>get_model_with_orbit</code>	<code>example_analysis.ipynb</code> Section 4, cell 22
<code>model_deltaChi2_jackknife</code>	there is no notebook demo yet — see <code>temp/jackknife_problem.py</code> at the repo root for a minimal driver
<code>model_likelihood</code>	<code>example_schwarmax.ipynb</code> cell 2 (defines wrapper), cell 5 (calls it to probe likelihood slices)
<code>get_rotation_curve</code>	<code>example_analysis.ipynb</code> Section 3, cell 15
<code>build_Lcirc_of_E</code>	<code>example_analysis.ipynb</code> Section 4, cell 23
<code>logMenc_logc_to_logM_logRs</code>	inside <code>unpack_params</code> — <code>example_analysis.ipynb</code> Section 0, cell 4

17. What to do when the user reports a bug

Before "fixing" anything:

1. Ask which pipeline generated the checkpoint (12-D vs 13-D — §14.1).
2. Confirm the parameter vector matches §4 slot-by-slot.
3. Check whether the user hand-crafted `dict_data` (they shouldn't; §5).
4. Check units of any custom `chi2_func` (§14.4).
5. Ask whether `t_t0_7` was pulled from LFS (§14.7).

Most reported "bugs" are unit or convention issues in the caller. Actual bugs in the package are rare — the code is short and heavily peer-reviewed via the example notebook.

18. Contact

If you get truly stuck, the source is small (~2,500 LOC of jax across the 12 package files) and every function has a short docstring or inline comment. `git log --oneline` shows a clean history; `git blame` each critical line to see who added it and why. Bar-branch commits from mid-2026 (this repo) are the most recent — earlier commits in the sibling `SchwarMAX/` codebase are archaeology and should be treated as such.

End of guide.